

// Q.1: Given a singly linked list having n nodes, write a program to find out the kth node from the end of the linked list, considering the following solution cases:

// (I) Without a hash table in a single pass/traversal of the linked list
// (II) Using a hash table

// (I): Without Hash Table – Single Pass Solution

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};
```

```
Node* findKthFromEnd(Node* head, int k) {
    Node* first = head;
    Node* second = head;

    // Move first k steps ahead
    for (int i = 0; i < k; i++) {
        if (!first) return nullptr; // if k > n
        first = first->next;
    }

    // Move both until first reaches end
    while (first) {
        first = first->next;
        second = second->next;
    }

    return second; // kth node from end
}
```

```
int main() {
    // Example Linked List: 10 -> 20 -> 30 -> 40 -> 50
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);
    head->next->next->next = new Node(40);
    head->next->next->next->next = new Node(50);

    int k = 2;
    Node* res = findKthFromEnd(head, k);

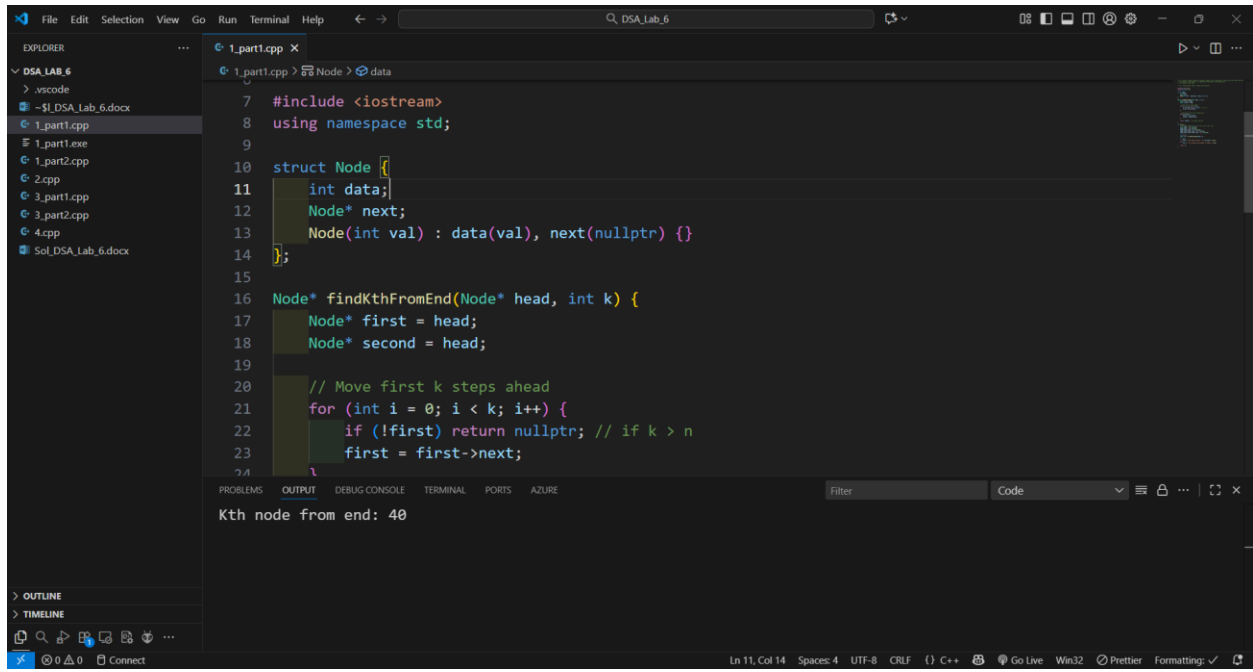
    if (res)
        cout << "Kth node from end: " << res->data << endl;
    else
```

```

        cout << "K is greater than number of nodes" << endl;

    return 0;
}

```



// (II): Using a Hash Table

```

#include <iostream>
#include <unordered_map>
using namespace std;

```

```

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

```

```

Node* findKthFromEndHash(Node* head, int k) {
    unordered_map<int, Node*> mp;
    Node* temp = head;
    int index = 0;

```

```

    // Store each node with index
    while (temp) {
        mp[index++] = temp;
        temp = temp->next;
    }

```

```

    int n = index; // total number of nodes

```

```

        if (k > n) return nullptr; // invalid k

        return mp[n - k];
    }

int main() {
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);
    head->next->next->next = new Node(40);
    head->next->next->next->next = new Node(50);

    int k = 3;
    Node* res = findKthFromEndHash(head, k);

    if (res)
        cout << "Kth node from end: " << res->data << endl;
    else
        cout << "K is greater than number of nodes" << endl;

    return 0;
}

```

The screenshot shows the Visual Studio Code interface with a C++ file named `1_part2.cpp` open. The code implements a function `findKthFromEndHash` that uses an unordered map to store nodes and their indices. The main function creates a linked list with nodes containing values 10, 20, 30, 40, and 50. It then calls `findKthFromEndHash` with `k=3`. The output window at the bottom shows the result: "Kth node from end: 30".

```

1 // (II): Using a Hash Table
2
3 #include <iostream>
4 #include <unordered_map>
5 using namespace std;
6
7 struct Node {
8     int data;
9     Node* next;
10    Node(int val) : data(val), next(nullptr) {}
11 };
12
13 Node* findKthFromEndHash(Node* head, int k) {
14     unordered_map<int, Node*> mp;
15     Node* temp = head;
16     int index = 0;
17
18     // Store each node with index
19
20     while (temp) {
21         mp[temp->data] = temp;
22         temp = temp->next;
23         index++;
24     }
25
26     if (index < k) return nullptr;
27
28     return mp[index - k + 1];
29 }
30
31 int main() {
32     Node* head = new Node(10);
33     head->next = new Node(20);
34     head->next->next = new Node(30);
35     head->next->next->next = new Node(40);
36     head->next->next->next->next = new Node(50);
37
38     int k = 3;
39     Node* res = findKthFromEndHash(head, k);
40
41     if (res)
42         cout << "Kth node from end: " << res->data << endl;
43     else
44         cout << "K is greater than number of nodes" << endl;
45
46     return 0;
47 }

```

Output: Kth node from end: 30

// Q.2: Write a program to find the cycle in a singly linked list using hashing.

```

#include <iostream>
#include <unordered_set>
using namespace std;

struct Node {

```

```

    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Function to detect cycle using hashing
bool detectCycle(Node* head) {
    unordered_set<Node*> visited;
    Node* temp = head;

    while (temp) {
        // If current node already seen, cycle exists
        if (visited.find(temp) != visited.end()) {
            return true;
        }

        // Mark current node as visited
        visited.insert(temp);
        temp = temp->next;
    }

    return false; // no cycle
}

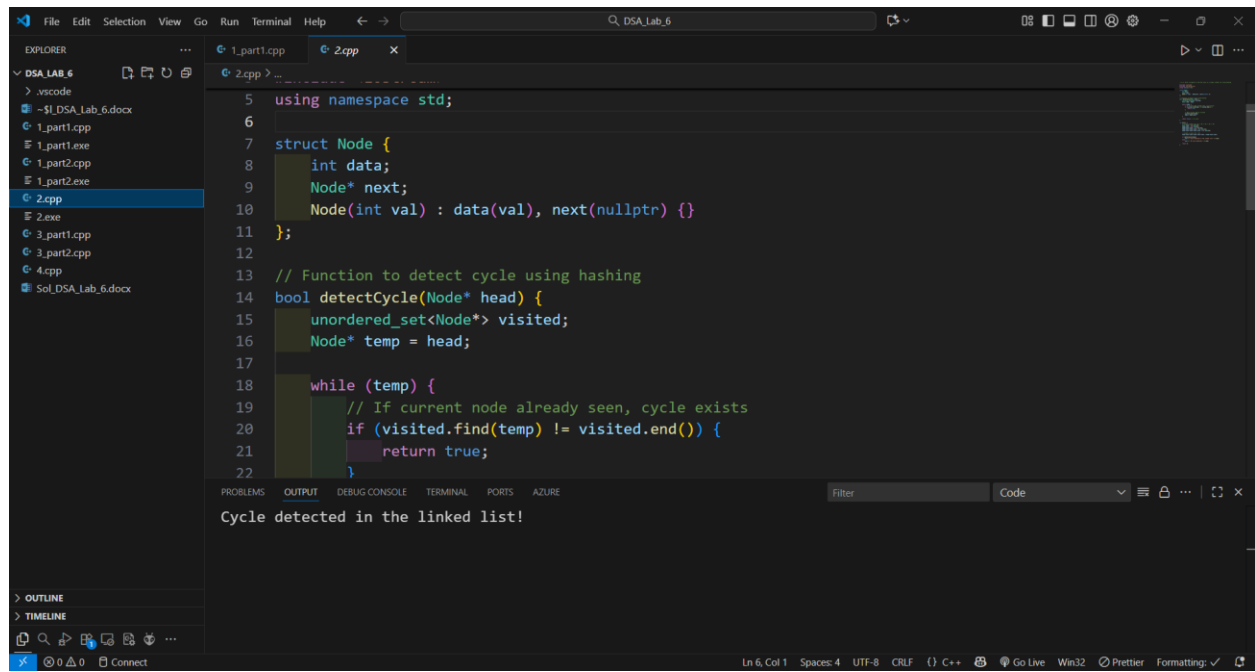
int main() {
    // Create linked list: 10 -> 20 -> 30 -> 40 -> 50
    Node* head = new Node(10);
    head->next = new Node(20);
    head->next->next = new Node(30);
    head->next->next->next = new Node(40);
    head->next->next->next->next = new Node(50);

    // Create a cycle: 50 -> 30
    head->next->next->next->next->next->next = head->next->next;

    if (detectCycle(head))
        cout << "Cycle detected in the linked list!" << endl;
    else
        cout << "No cycle detected." << endl;

    return 0;
}

```



// Q.3: Given a singly linked list having integer values, write a program to remove the duplicate values from the linked list, considering the following solution cases:
// (I) Without using a hash table (not sorting)
// (II) Using a hash table
// e.g., input values of the nodes in a linked list: 5 6 3 3 8 5 6 9 7 2 3, output: 5 6 3 8 9 7 2

```

// (I) Without using a hash table (not sorting)
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Function to remove duplicates without hash table
void removeDuplicates_NoHash(Node* head) {
    Node* current = head;

    while (current != nullptr) {
        Node* runner = current;
        while (runner->next != nullptr) {
            if (runner->next->data == current->data) {
                // Duplicate found - remove it
                Node* dup = runner->next;
                runner->next = runner->next->next;
            }
            runner = runner->next;
        }
        current = current->next;
    }
}

```

```

        delete dup;
    } else {
        runner = runner->next;
    }
}
current = current->next;
}

// Utility function to print the linked list
void printList(Node* head) {
    Node* temp = head;
    while (temp) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Input: 5 6 3 3 8 5 6 9 7 2 3
    Node* head = new Node(5);
    head->next = new Node(6);
    head->next->next = new Node(3);
    head->next->next->next = new Node(3);
    head->next->next->next->next = new Node(8);
    head->next->next->next->next->next = new Node(5);
    head->next->next->next->next->next->next = new Node(6);
    head->next->next->next->next->next->next->next = new Node(9);
    head->next->next->next->next->next->next->next->next = new
Node(2);
    head->next->next->next->next->next->next->next->next->next = new
Node(3);

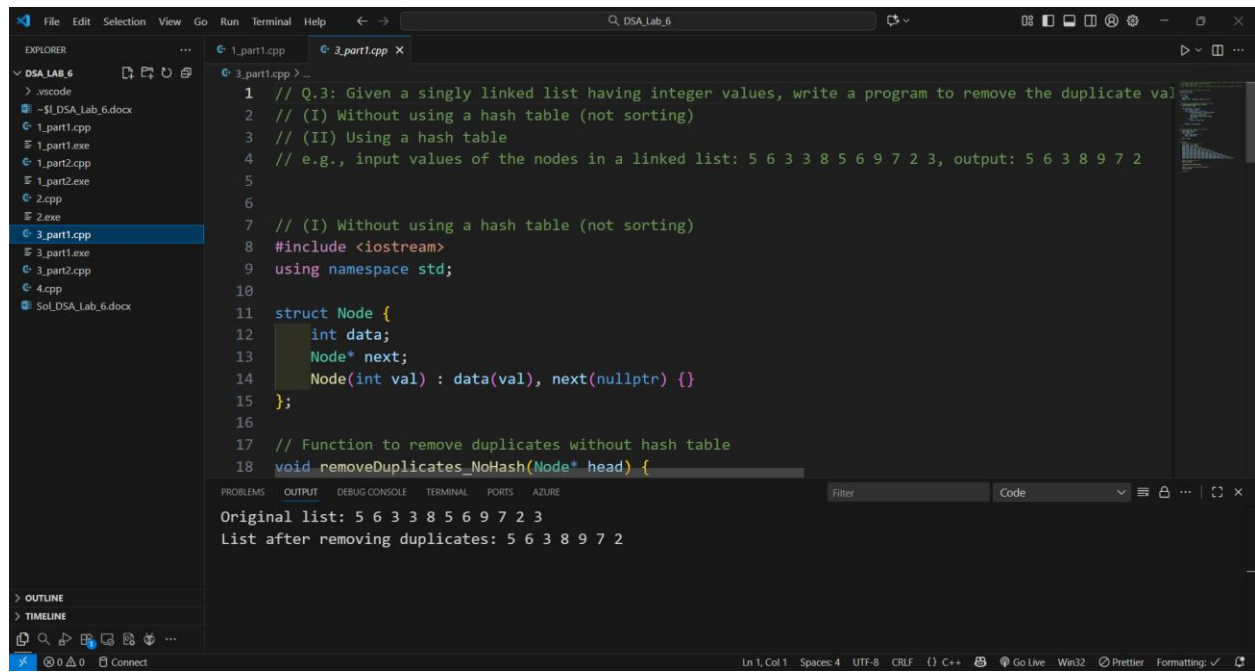
    cout << "Original list: ";
    printList(head);

    removeDuplicates_NoHash(head);

    cout << "List after removing duplicates: ";
    printList(head);

    return 0;
}

```



```
// (II) Using a hash table
#include <iostream>
#include <unordered_set>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

// Function to remove duplicates using hash table
void removeDuplicates_Hash(Node* head) {
    unordered_set<int> seen;
    Node* current = head;
    Node* prev = nullptr;

    while (current != nullptr) {
        if (seen.find(current->data) != seen.end()) {
            // Duplicate found - remove node
            prev->next = current->next;
            delete current;
        } else {
            seen.insert(current->data);
            prev = current;
        }
        current = prev->next;
    }
}
```

```

// Utility function to print linked list
void printList(Node* head) {
    Node* temp = head;
    while (temp) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

int main() {
    // Input: 5 6 3 3 8 5 6 9 7 2 3
    Node* head = new Node(5);
    head->next = new Node(6);
    head->next->next = new Node(3);
    head->next->next->next = new Node(3);
    head->next->next->next->next = new Node(8);
    head->next->next->next->next->next = new Node(5);
    head->next->next->next->next->next->next = new Node(6);
    head->next->next->next->next->next->next->next = new Node(9);
    head->next->next->next->next->next->next->next->next = new Node(7);
    head->next->next->next->next->next->next->next->next->next = new
Node(2);
    head->next->next->next->next->next->next->next->next->next->next = new
Node(3);

    cout << "Original list: ";
    printList(head);

    removeDuplicates_Hash(head);

    cout << "List after removing duplicates: ";
    printList(head);

    return 0;
}

```

```

5
6 struct Node {
7     int data;
8     Node* next;
9     Node(int val) : data(val), next(nullptr) {}
10 };
11
12 // Function to remove duplicates using hash table
13 void removeDuplicates_Hash(Node* head) {
14     unordered_set<int> seen;
15     Node* current = head;
16     Node* prev = nullptr;
17
18     while (current != nullptr) {
19         if (seen.find(current->data) != seen.end()) {
20             // Duplicate found - remove node
21             prev->next = current->next;
22             delete current;

```

Original list: 5 6 3 3 8 5 6 9 7 2
List after removing duplicates: 5 6 3 8 9 7 2

// Q.4: Given a list of integer pairs, where first element of
 // each pair is unique, write a program to detect the symmetric pairs,
 // e.g., given list {(10,20), (5,6), (8,9), (20,10), (2,3), (3,1)} has
 // (10, 20) and (20,10)
 // symmetric pairs. [Use hashing to solve the problem.]

```

#include <iostream>
#include <unordered_map>
#include <vector>
using namespace std;

// Function to find symmetric pairs
void findSymmetricPairs(vector<pair<int, int>> pairs) {
    unordered_map<int, int> mp;

    for (auto p : pairs) {
        int first = p.first;
        int second = p.second;

        // Check if symmetric pair exists (second, first)
        if (mp.find(second) != mp.end() && mp[second] == first) {
            cout << "Symmetric pair found: (" << first << ", " << second
                << ") and (" << second << ", " << first << ")" << endl;
        } else {
            mp[first] = second; // store current pair
        }
    }
}

int main() {

```

```

vector<pair<int, int>> pairs = {
    {10, 20},
    {5, 6},
    {8, 9},
    {20, 10},
    {2, 3},
    {3, 1}
};

findSymmetricPairs(pairs);

return 0;
}

```

The screenshot shows a Visual Studio Code editor window with a C++ file named `4.cpp` open. The code defines a function `findSymmetricPairs` that takes a vector of pairs and uses an `unordered_map` to find symmetric pairs. The output window at the bottom shows the result of the function call.

```

6  #include <iostream>
7  #include <vector>
8  #include <unordered_map>
9  using namespace std;
10
11 // Function to find symmetric pairs
12 void findSymmetricPairs(vector<pair<int, int>> pairs) {
13     unordered_map<int, int> mp;
14
15     for (auto p : pairs) {
16         int first = p.first;
17         int second = p.second;
18
19         // Check if symmetric pair exists (second, first)
20         if (mp.find(second) != mp.end() && mp[second] == first) {
21             cout << "Symmetric pair found: (" << first << ", " << second
22             << ") and (" << second << ", " << first << ")" << endl;
23         } else {
24             mp[first] = second; // store current pair
25         }
26     }
27 }

```

The output window displays the following message:

```
Symmetric pair found: (20, 10) and (10, 20)
```